

Empowering LLMs for Structure-Based Drug Design via Exploration-Augmented Latent Inference

Xuanning Hu
College of Computer Science and
Technology, Jilin University
Key Laboratory of Symbolic
Computation and Knowledge
Engineering of Ministry of Education,
Jilin University
Changchun, Jilin, China
huxn24@mails.jlu.edu.cn

Anchen Li
Department of Computer Science,
Aalto University
Espoo, Finland
anchen.li@aalto.fi

Qianli Xing*
College of Computer Science and
Technology, Jilin University
Key Laboratory of Symbolic
Computation and Knowledge
Engineering of Ministry of Education,
Jilin University
Changchun, Jilin, China
qianlixing@jlu.edu.cn

Jinglong Ji
College of Artificial Intelligence, Jilin
University
Key Laboratory of Symbolic
Computation and Knowledge
Engineering of Ministry of Education,
Jilin University
Changchun, Jilin, China
jjjl22@mails.jlu.edu.cn

Hao Tuo
College of Computer Science and
Technology, Jilin University
Key Laboratory of Symbolic
Computation and Knowledge
Engineering of Ministry of Education,
Jilin University
Changchun, Jilin, China
tuohao25@mails.jlu.edu.cn

Bo Yang*
College of Computer Science and
Technology, Jilin University
Key Laboratory of Symbolic
Computation and Knowledge
Engineering of Ministry of Education,
Jilin University
Changchun, Jilin, China
ybo@jlu.edu.cn

Abstract

Large Language Models (LLMs) possess strong representation and reasoning capabilities, but their application to structure-based drug design (SBDD) is limited by insufficient understanding of protein structures and unpredictable molecular generation. To address these challenges, we propose Exploration-Augmented Latent Inference for LLMs (ELILLM), a framework that reinterprets the LLM generation process as an encoding, latent space exploration, and decoding workflow. ELILLM explicitly explores portions of the design problem beyond the model's current knowledge while using a decoding module to handle familiar regions, generating chemically valid and synthetically reasonable molecules. In our implementation, Bayesian optimization guides the systematic exploration of latent embeddings, and a position-aware surrogate model efficiently predicts binding affinity distributions to inform the search. Knowledge-guided decoding further reduces randomness and effectively imposes chemical validity constraints. We demonstrate ELILLM on the CrossDocked2020 benchmark, showing strong controlled exploration and high binding affinity scores compared with seven baseline methods. These results demonstrate that ELILLM can effectively enhance LLMs' capabilities for SBDD. Our code is available at <https://github.com/hxn/hxn/ELILLM>.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792605>

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Search methodologies*; • **Applied computing** → **Computational biology**.

Keywords

Structure-Based Drug Design, Large Language Models, Latent Space, Bayesian Optimization

ACM Reference Format:

Xuanning Hu, Anchen Li, Qianli Xing, Jinglong Ji, Hao Tuo, and Bo Yang. 2026. Empowering LLMs for Structure-Based Drug Design via Exploration-Augmented Latent Inference. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792605>

1 Introduction

Structure-Based Drug Design (SBDD), which aims to design novel compounds conditioned on structural information specific to the protein binding site, is a fundamental problem in drug discovery and biomedical research, and has been widely applied in tasks such as virtual screening and lead compound identification. Early approaches adopted variational autoencoders (VAEs) [5, 26], autoregressive models [21, 23, 25], and more recently, diffusion models [9–11, 13]—all of which have shown strong performance in generating valid and diverse molecules. However, unlike human experts who iteratively refine molecular designs based on accumulated feedback, these models typically follow a generation process that lacks intermediate feedback and relies heavily on the training data distribution, which makes their generation less flexible and prevents them from dynamically incorporating new experience.

Large Language Models (LLMs), with their strong generalization and reasoning abilities, offer a promising way to address these limitations. Equipped with good understanding of natural language, strong representation skills, and flexible generation abilities, LLMs are suitable for step-by-step molecular design guided by intermediate feedback. This allows more intuitive interactions and the possibility to follow expert-like, iterative design processes, making AI-assisted drug discovery easier to use. Their success in complex reasoning tasks such as code generation [27], theorem proving [34], and strategic planning [32] shows that these abilities can also be applied to more interactive molecular design.

However, LLMs face significant challenges in SBDD. Shen et al. [29] point out that LLMs do not have enough understanding of protein sequences, likely because their pretraining data does not include enough relevant information. Kristiadi et al. [15] report that LLMs are less accurate than traditional models in predicting molecular properties. These results suggest that current LLMs are unlikely to generate target molecules reliably. Brahmavar et al. [4] tried to guide LLMs for targeted generation using logical feedback, but the generated molecules were often similar to the input compounds and appeared almost random, showing weak targeting performance. To address this problem, some studies avoid using general LLMs and instead train specialized chemical GPT models on large protein–ligand datasets [35, 37, 42]. However, this approach reduces the natural language capabilities of LLMs and still faces the same limitations as traditional deep learning models, being restricted by the distribution of training data. Therefore, finding effective ways to guide LLMs for SBDD remains an important and meaningful research direction.

To this end, we propose **Exploration-Augmented Latent Inference for Large Language Models (ELILLM)**, a framework that reinterprets the LLM generation process as an encoding, exploration, and decoding workflow for SBDD. ELILLM leverages the latent space of LLMs to explicitly explore portions of the design problem that fall outside the model’s current understanding, such as generating molecules with optimized binding to the target protein, while allowing the model to handle familiar regions effectively through the decoding module, producing chemically valid and synthetically reasonable molecules. Specifically, Bayesian optimization is applied in the latent space to systematically explore latent embeddings, enhancing the LLM’s ability to generate molecules aligned with the structural information of the target protein. To handle variable-length embeddings in the latent space, we introduce a *position-aware surrogate model*. By using a position-invariant aggregation strategy, this model efficiently predicts the affinity distribution corresponding to latent embeddings, guiding exploration in the latent space. Finally, we develop a *knowledge-guided LLM decoding* strategy that repairs candidate embeddings using chemical knowledge, reducing the randomness in LLM decoding and producing ligand molecules that satisfy task-specific requirements. Our main contributions are summarized as follows.

- We propose a novel framework, **ELILLM**, that enhances LLMs for structure-based drug design by explicitly exploring the latent space. The framework reinterprets the LLM generation process as an encoding, exploration, and decoding workflow, allowing targeted exploration of regions outside the

LLM’s current understanding while leveraging its strengths for familiar regions. The workflow is clearly defined, making it easy to apply ELILLM to SBDD tasks.

- We implement this framework for SBDD using a Bayesian optimization-based exploration strategy. The approach consists of three main processes: Molecular-Guided LLM Encoding, BO-Based Latent Space Exploration, and Knowledge-Guided LLM Decoding. This implementation demonstrates how ELILLM can efficiently simulate expert-like iterative molecular design with interpretability and control.
- We perform a comprehensive evaluation on the CrossDocked2020 benchmark, ensuring consistent Vina docking scores for identical SMILES. The results show that ELILLM improves LLMs’ ability to perform controlled exploration in the latent space, generating molecules with higher predicted binding affinity. Our method achieves strong performance, surpassing seven baseline approaches, highlighting its effectiveness for challenging SBDD tasks.

2 Methodology

In this section, we describe the framework of ELILLM, which enables exploration in the latent space of LLM. The structure of this framework is illustrated in Figure 1 (A). To address SBDD tasks, we formulate the generation process of LLM as three main processes, including encoding, exploring, and decoding. Firstly, the encoding process is formulated as f_{enc} :

$$f_{\text{enc}}(s) = E[\text{tokenizer}(s)], \quad f_{\text{enc}} : \text{String} \rightarrow \mathbb{R}^{l \times d}. \quad (1)$$

The $\text{tokenizer}(s) \in \mathbb{N}^l$ is the tokenized input sequence, and $E \in \mathbb{R}^{V \times d}$ is the embedding matrix that maps tokens to d -dimensional vectors. Then, we introduce the latent-space exploring process f_{explore} , which can systematically explore a continuum of possible entities with desired properties. It is defined as:

$$f_{\text{explore}}(\cdot) : \mathcal{X} \rightarrow \mathbb{R}^{l' \times d}, \quad (2)$$

where $\mathcal{X}$ denotes an unconstrained input space that can include prompt embeddings, prior knowledge, task-specific conditions, or any other information source. The only requirement is that the output of f_{explore} lies in the LLM’s latent space, i.e., it must be a valid embedding in $\mathbb{R}^{l' \times d}$. Since this process transforms one latent embedding into another through exploration and effectively performs a step of reasoning in the latent space, we refer to it as *exploration-augmented latent inference*. Such embeddings can then be directly consumed by the decoder f_{dec} for autoregressive generation. Next, the decoding process is denoted by f_{dec} as follows:

$$f_{\text{dec}}(z) = \text{LLM}(z), \quad f_{\text{dec}} : \mathbb{R}^{l \times d} \rightarrow \text{String}. \quad (3)$$

The decoding process represents the autoregressive generation process of an LLM. f_{dec} consumes the embedding vectors $z \in \mathbb{R}^{l \times d}$, and proceeds with autoregressive generation one token at a time, until an end-of-sequence token is produced. Overall, a single iteration of the ELILLM framework can be formalized as:

$$\text{ELILLM}(\text{prompt}) = f_{\text{dec}}(f_{\text{explore}}(f_{\text{enc}}(\text{prompt}), \star)), \quad (4)$$

where $\star$ denotes any additional input that conditions the exploration process, such as task-specific prior knowledge, optimization

signals, or structural constraints. The decoder f_{dec} directly consumes the latent representation produced by f_{explore} , and generates an output sequence autoregressively.

3 Method Details

In this section, we present the concrete implementation of the ELILLM framework. The detailed modeling based on the ELILLM framework are provided in the following.

3.1 Problem Definition

SBDD plays a pivotal role in drug discovery, aiming to create ligand molecules with high affinity for a specific protein binding pocket. In this work, we use $s \in \mathbb{S}$ to denote a ligand molecule and $t \in \mathbb{T}$ to represent the target protein and its binding pocket. To evaluate the binding affinity between s and t , we employ a black-box docking score function $\text{Dock} : \mathbb{S} \times \mathbb{T} \rightarrow \mathbb{R}$, which leverages the 3D structural information. A lower docking score indicates a stronger binding affinity. Given an observed dataset denoted as:

$$\mathcal{D}_{\text{obs}} = \{(s_i, y_i)\}_{i=1}^N, \quad (5)$$

where each $s_i \in \mathbb{S}$ is a symbolic molecular sequence and $y_i = \text{Dock}(s_i, t)$ is its associated docking score, our objective is to find a new subset $\mathcal{S}_{\text{top}} \subseteq \mathbb{S} \setminus \{s_i\}_{i=1}^N$ of size k :

$$\mathcal{S}_{\text{top}} = \underset{S' \subseteq \mathbb{S}_{\text{rem}}}{\text{argmin}} \sum_{s \in S'} \text{Dock}(s, t), \quad (6)$$

where $\mathbb{S}_{\text{rem}} = \mathbb{S} \setminus \{s_i\}_{i=1}^N$ and $|S'| = k$.

3.2 Overview

In the SBDD task, we model three processes of the ELILLM framework, which is depicted in Figure 1(B). The process of Molecular-Guided LLM Encoding encodes observed molecules into the semantic latent space of the LLM, resulting in their corresponding latent embeddings. The process of BO-based Exploration in LLM Latent Embedding Space aims to identify embeddings that are expected to yield improved docking scores, which leverages the advantage of Bayesian optimization in efficiently searching black-box objectives under limited samples. It consists of three submodels, including Latent Space Sampling Strategy Model, Position-Aware Surrogate Model and Acquisition Function Model. Specifically, the Latent Space Sampling Strategy Model perturbs known molecular embeddings with multiplicative Gaussian noise to progressively generate a large set of exploratory points in each round based on the idea of spatial locality. The Position-Aware Surrogate Model employs a position-aware aggregation mechanism that is capable of handling variable-length embedding sequences while perceiving positional information, and efficiently computes the predictive distribution of binding affinities. Finally, the Acquisition Function Model exploits the predictive distribution to balance exploitation and exploration, selecting candidate embeddings with high exploration potential. The process of Knowledge-Guided LLM Decoding guides the LLM to perform robust decoding by incorporating basic chemical knowledge and defining its role as a SMILES repair engine. Each candidate embedding is decoded into a valid molecular SMILES, evaluated using the black-box docking score, and added to the observed data to complete one iteration. The pseudocode is provided in Appendix A.

3.3 Molecular-Guided LLM Encoding

We first introduce the construction of observed dataset. For targets with known high-affinity ligands, existing ligands can be directly utilized. In novel drug discovery scenarios, we may either randomly sample molecules from existing chemical libraries or employ existing deep generative models as tools to produce initial ligand candidates that satisfy basic requirements for the given target protein.

Given the observed dataset $\mathcal{D}_{\text{obs}} = \{(s_i, y_i)\}_{i=1}^N$, we compute latent embeddings $z_i = f_{\text{enc}}(s_i) \in \mathbb{R}^{n_i \times d}$, where n_i denotes the length of the token sequence corresponding to s_i , and d is the hidden dimension of the embedding space. The set of latent embeddings of observed molecules is denoted as:

$$\mathcal{Z}_{\text{obs}} = \{z_i\}_{i=1}^N. \quad (7)$$

3.4 BO-based Exploration

3.4.1 Latent Space Sampling Strategy. To enable local exploration in the latent space, we apply multiplicative Gaussian perturbations to each z_i , generating perturbed embeddings via element-wise noise:

$$\tilde{z}_i^{(j)} = z_i \odot \epsilon_i^{(j)}, \epsilon_i^{(j)} \sim \mathcal{N}(1, \lambda_{\text{perturb}})^{n_i \times d}, \quad (8)$$

where $j = 1, \dots, M$. λ_{perturb} is a tunable hyperparameter that controls the magnitude of perturbation. A larger λ_{perturb} results in wider exploration around the original embedding, while a smaller value enforces more localized sampling. The set of perturbed embeddings is denoted by:

$$\mathcal{Z}_{\text{explore}} = \left\{ \tilde{z}_i^{(j)} \mid i = 1, \dots, N; j = 1, \dots, M \right\}. \quad (9)$$

3.4.2 Position-Aware Surrogate Model. While recent studies have proposed expressive molecular representation learning methods based on graph neural networks [17, 18], these approaches are primarily designed for supervised or self-supervised learning on explicit molecular graphs. In contrast, our surrogate model operates directly on the latent representations of a pretrained LLM and must be trained online with a limited number of evaluated samples, where reliable uncertainty estimation and sample efficiency are critical. As a result, we adopt a lightweight, position-aware surrogate that aligns with the requirements of iterative black-box optimization in latent space.

A key requirement for the surrogate model is to provide well-calibrated uncertainty estimates, which are essential for balancing exploration and exploitation during latent space optimization. Gaussian Processes (GPs) [40] are a natural choice due to their principled Bayesian formulation and inherent uncertainty quantification. However, modern LLMs such as LLaMA 3.1 [8] typically produce high-dimensional molecular embeddings (e.g., $d = 4096$), which pose significant challenges for standard GPs. In such high-dimensional spaces, data sparsity severely degrades kernel expressiveness, leading to unreliable predictive uncertainty [24].

To address these challenges, we employ a Deep Kernel Gaussian Process (DKGP) [41] as our surrogate model. DKGP integrates a learnable neural projection that maps high-dimensional LLM embeddings into a lower-dimensional, task-adaptive feature space, enabling effective uncertainty-aware modeling while preserving the Bayesian advantages of GPs. This design allows the surrogate to

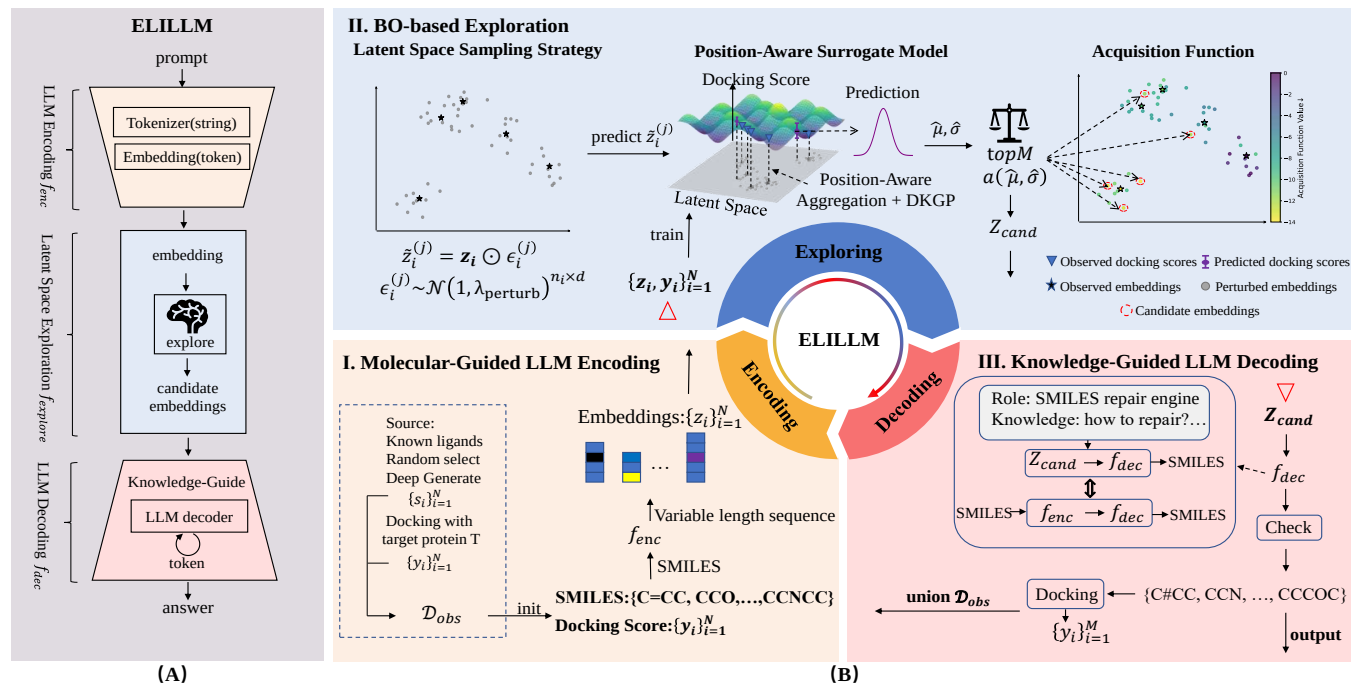


Figure 1: Overview of the ELILLM framework. (A) The abstract framework of ELILLM. (B) A concrete instantiation of ELILLM, illustrating how the framework operates in a single iteration. Specifically, molecules in the observed dataset $\mathcal{D}_{obs}$ are first encoded into embeddings. Next, the Latent Space Sampling Strategy samples a large set of candidate points to be evaluated. Simultaneously, the Position-Aware Surrogate Model is trained using the molecular embeddings as inputs and the corresponding docking scores as labels. The trained surrogate model is then used to predict the docking score distribution of the candidate points, and the acquisition function balances exploitation and exploration to select the most promising embeddings. Finally, through Knowledge-Guided LLM Decoding, chemical knowledge is used to constrain the LLM’s decoding behavior, converting candidate embeddings into the closest valid molecular SMILES to form a candidate molecule set. The candidate molecules are evaluated with black-box docking software, and the resulting molecule–score pairs are incorporated back into $\mathcal{D}_{obs}$.

remain both expressive and sample-efficient under the constraints of online latent-space optimization.

Additionally, each molecular embedding $\tilde{z}_i^{(j)} \in \mathbb{R}^{n_i \times d}$ is a variable-length sequence, where n_i differs across molecules. Although models like Transformers and LSTMs are capable of directly processing such sequences, their computational demands become impractical for the evaluation of thousands of candidates in iterative searches.

To preserve positional information during aggregation, we compute a position-aware embedding as:

$$\tilde{z}_i^{(j)} = \frac{1}{n_i} \sum_{t=1}^{n_i} \frac{\text{concat}(p_t \tilde{z}_{i,t}^{(j)}, (l_{\max} - p_t) \tilde{z}_{i,t}^{(j)})}{l_{\max}}, \quad (10)$$

where p_t denotes the positional index of the token embedding $\tilde{z}_{i,t}^{(j)}$, and $l_{\max}$ is the predefined maximum number of generated tokens per molecule. Thus, $\tilde{z}_i^{(j)}$ encodes both the absolute and relative position of each token, which can enhance the surrogate model’s ability to capture structural information in the latent space.

Our position-aware aggregation introduces a novel form of position sensitivity by reweighting token embeddings according to their positions. Moreover, under our setting where token embeddings are obtained without positional encoding (Theorem 3.1), the

embeddings are independent of their positions. Combined with the proposed aggregation operation, this ensures that the resulting aggregated embeddings are positionally unbiased (Theorem 3.2). The detailed proofs are provided in Appendix A.

THEOREM 3.1 (INDEPENDENCE OF TOKEN EMBEDDINGS AND POSITIONS). Let $s = (x_1, x_2, \dots, x_l)$ be a token sequence, and let $\text{tokenizer}(s)$ map s to discrete token indices $(t_1, t_2, \dots, t_l)$. Let E be an embedding matrix mapping each token t to a vector $E[t] \in \mathbb{R}^d$, and define the token embeddings as

$$\tilde{z}_t = E[t].$$

If no positional encoding is added (e.g., no sinusoidal or learned positional embedding), then for each sequence s the embedding $\tilde{z}_t$ of token t is independent of its positional index $p_t \in \{1, \dots, l\}$.

THEOREM 3.2 (POSITIONAL UNBIASEDNESS OF AGGREGATION). Assume the token embeddings $\tilde{z}_{i,t}^{(j)}$ are obtained via a tokenizer and embedding matrix without any positional encoding, so that by Theorem 3.1, they are independent of their positional indices p_t . Then, the position-aware aggregation defined in Eq. (10) is statistically unbiased with respect to position, i.e., the expected aggregated embedding $\mathbb{E}[\tilde{z}_i^{(j)}]$ treats all token positions equally.

The aggregated embedding $\tilde{z}_i^{(j)} \in \mathbb{R}^{2d}$ is then passed through a multilayer perceptron (MLP), yielding a transformed feature vector:

$$\hat{z}_i^{(j)} = f_{\text{MLP}}(\tilde{z}_i^{(j)}). \quad (11)$$

We denote this overall transformation as $\phi_\theta(\cdot)$, including the position-aware aggregation and dimensionality-reducing through an MLP. $\phi_\theta(\cdot)$ maps a variable-length embedding sequence to a fixed-length vector: $\phi_\theta: \mathbb{R}^{n_i \times d} \rightarrow \mathbb{R}^{d'}$, where d is the embedding dimension (e.g., 4096) and d' is a predefined lower dimension (e.g., 20). The GP is defined over the transformed inputs, giving the surrogate model:

$$\hat{f}_\theta(z) \sim \mathcal{GP}(0, k_{\text{GP}}(\phi_\theta(z), \phi_\theta(z'))), \quad (12)$$

where k_{GP} is the learnable kernel function in the transformed space and the detailed construction of the kernel function is provided in Appendix B. For any explored embeddings $\tilde{z}_i^{(j)}$, the model produces a predictive distribution with mean $\hat{\mu}(\tilde{z}_i^{(j)})$ and variance $\hat{\sigma}^2(\tilde{z}_i^{(j)})$, representing the predicted docking score and the model uncertainty, respectively:

$$\hat{f}_\theta(\tilde{z}_i^{(j)}) \sim \mathcal{N}(\hat{\mu}(\tilde{z}_i^{(j)}), \hat{\sigma}^2(\tilde{z}_i^{(j)})) \quad (13)$$

3.4.3 Hierarchical Training Strategy. We adopt a hierarchical training strategy for the surrogate model. The training dataset $\mathcal{D}_{\text{train}} = \{(z_i, y_i)\}_{i=1}^N$ is constructed from the latent embeddings $\mathcal{Z}_{\text{obs}}$ and their associated docking scores.

In the first stage, we train a composite predictor that maps each latent input z_i to a scalar prediction using a transformation ϕ_θ followed by a regression head h . The entire model is optimized by minimizing the mean squared error loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (h(\phi_\theta(z_i)) - y_i)^2. \quad (14)$$

After convergence, the regression head is discarded and ϕ_θ is frozen. In the second stage, a Gaussian process is trained on the transformed embeddings by minimizing the negative log marginal likelihood:

$$\mathcal{L}_{\text{GP}} = -\log p(\mathbf{y} | \phi_\theta(\mathbf{Z})), \quad (15)$$

where $\mathbf{Z} = \mathcal{Z}_{\text{obs}}$ and $\mathbf{y} = \{y_i\}_{i=1}^N$.

This two-stage design decouples representation learning from uncertainty modeling, allowing the model to first focus on effective feature extraction and then on principled uncertainty quantification. Such separation enhances training stability and improves predictive performance, particularly under the data-scarce conditions commonly encountered in surrogate modeling.

3.4.4 Acquisition Function. We adopt the lower confidence bound (LCB) acquisition function to guide exploration in the latent space. This acquisition function balances the predictive mean and uncertainty of the surrogate model, and is defined as:

$$\alpha_{\text{LCB}}(z) = \hat{\mu}(z) - \kappa_t \cdot \hat{\sigma}(z), \quad (16)$$

where $\hat{\mu}(z)$ and $\hat{\sigma}(z)$ denote the predictive mean and standard deviation, respectively, predicted by the surrogate model given the latent input z .

We follow the schedule proposed by Srinivas et al. [30], due to its strong theoretical guarantees in the Bayesian optimization setting.

Specifically, the exploration coefficient κ_t is dynamically updated as:

$$\kappa_t = \sqrt{2 \log \left(\frac{t^2 \pi^2}{6\delta} \right)}, \quad (17)$$

where $\delta \in (0, 1)$ is a confidence parameter. In our setup, we set $t = |\mathcal{D}_{\text{train}}| + 1$, such that the exploration strength naturally increases with the amount of observed data.

After computing the acquisition scores for all perturbed latent embeddings $\mathcal{Z}_{\text{explore}}$, we select the top candidates for further evaluation. Specifically, we choose the n_{cand} latent embeddings with the lowest acquisition scores, forming the candidate set:

$$\mathcal{Z}_{\text{cand}} = \underset{z \in \mathcal{Z}_{\text{explore}}}{\text{argmin_top}}^{n_{\text{cand}}} \alpha_{\text{LCB}}(z). \quad (18)$$

3.5 Knowledge-Guided LLM Decoding

The current problem is how to design the LLM decoding function f_{dec} that transforms $\mathcal{Z}_{\text{cand}}$ into valid ligand molecules. However, reliably implementing this function in practice presents significant challenges. Due to the randomness of LLM generation, the same input can yield different outputs if not properly constrained, leading to instability in decoding. Moreover, since candidate embeddings are derived via E.q. 8, they may partially lie outside the valid chemical embedding space $\mathcal{Z}_s$, increasing the likelihood of producing invalid or off-target molecules.

To address these issues, we employ knowledge-guided, role-based prompting to guide the LLM. First, we define the LLM’s role as a SMILES repair engine, prompting it to minimally adjust candidate embeddings into chemically plausible structures. Building on this, we provide chemically informed instructions (e.g. complete missing ring numbers: C1CC $\rightarrow$ C1CC1) to handle common error types, further constraining the generation process. The entire decoding process is formulated as follows:

$$\mathcal{S}_{\text{cand}} = \{f_{\text{dec}}(z, \text{prompt}) \mid z \in \mathcal{Z}_{\text{cand}}\}, \quad (19)$$

where $\mathcal{S}_{\text{cand}}$ denotes the set of SMILES strings generated from the latent candidates. Afterwards, we will store $\mathcal{S}_{\text{cand}}$ and their docking scores in $\mathcal{D}_{\text{obs}}$ to guide the generation of subsequent iterations.

$$\mathcal{D}_{\text{obs}} \leftarrow \mathcal{D}_{\text{obs}} \cup \{(s, \text{Dock}(s, t)) \mid s \in \mathcal{S}_{\text{cand}}\} \quad (20)$$

4 Experiments

In this section, we evaluate our ELILLM on the SBDD task to answer two main questions: (Q1) can the ELILLM framework achieve the expected generation performance on domain-specific tasks? (Q2) how does ELILLM perform efficient exploration in high-affinity regions? In addition, we conduct ablation studies and visualization experiments to further demonstrate the effectiveness of our framework. Owing to space constraints, supplementary experiments (e.g., the Wilcoxon signed-rank test) are included in Appendix D.

4.1 Experiment Setup

4.1.1 Dataset and Settings. We evaluate ELILLM using the Cross-Docked2020 dataset[7]. In line with standard practice [9, 42], the dataset includes 65K drug–target pairs for training and 100 target proteins for testing. Unlike most methods that train on the entire training set, we consider two settings for constructing the observed

Table 1: Evaluation of generated molecules is primarily based on the Vina docking score (Top 1, 5, 10, and 20), with *QED*, *SA*, and *diversity* serving as auxiliary metrics. We report both average (Avg.) and median (Med.) values across targets; for most metrics, Med. denotes the mean of per-target medians, while for diversity—yielding one value per target—Med. denotes the median of the 100 target-level scores. The best and second-best results are highlighted in bold and underlined, respectively.

Methods	Top1 Dock ↓		Top5 Dock ↓		Top10 Dock ↓		Top20 Dock ↓		<i>QED</i>		<i>SA</i>		<i>Diversity</i>	
	Avg.	Med.	Avg.	Med.	Avg.	Med.	Avg.	Med.	Avg.	Med.	Avg.	Med.	Avg.	Med.
Random Init	-8.62	-8.62	-8.19	-8.14	-7.90	-7.81	-6.38	-7.42	0.53	0.53	0.76	0.77	0.78	0.77
AR (2021)	-8.41	-8.41	-8.08	-8.03	-7.88	-7.82	-7.64	-7.56	0.51	0.50	0.63	0.63	0.70	0.70
Pocket2Mol (2022)	-9.15	-9.15	-8.85	-8.82	-8.64	-8.60	-8.38	-8.32	0.56	0.57	0.74	0.75	0.69	0.71
liGAN (2022)	-8.13	-8.13	-7.78	-7.73	-7.58	-7.53	-7.34	-7.26	0.39	0.39	0.59	0.57	0.66	0.67
TargetDiff (2023)	<u>-9.38</u>	<u>-9.38</u>	-8.86	-8.78	-8.53	-8.43	-8.13	-8.00	0.48	0.48	0.58	0.58	0.72	0.71
ALIDIFF (2024)	-9.37	-9.37	-8.85	-8.80	-8.52	-8.40	-8.11	-8.01	0.50	0.50	0.57	0.56	0.73	0.71
TamGen (2024)	-8.53	-8.53	-8.11	-8.05	-7.72	-7.65	-7.70	-7.62	0.56	0.56	0.77	0.78	0.75	0.74
LMLF-rand (2024)	-8.66	-8.66	-8.17	-8.10	-7.87	-7.78	-7.50	-7.40	0.57	0.59	0.77	0.79	0.81	0.80
LMLF-diff (2024)	-9.05	-9.05	-8.52	-8.43	-8.19	-8.11	-7.79	-7.67	0.53	0.53	0.64	0.63	0.78	0.77
ELILLM-rand	-9.33	-9.33	<u>-8.98</u>	<u>-8.93</u>	<u>-8.74</u>	<u>-8.68</u>	<u>-8.44</u>	<u>-8.36</u>	0.46	0.45	0.63	0.63	0.69	0.70
ELILLM-diff	-9.80	-9.80	-9.37	-9.31	-9.09	-9.03	-8.74	-8.65	0.49	0.49	0.57	0.56	0.67	0.68

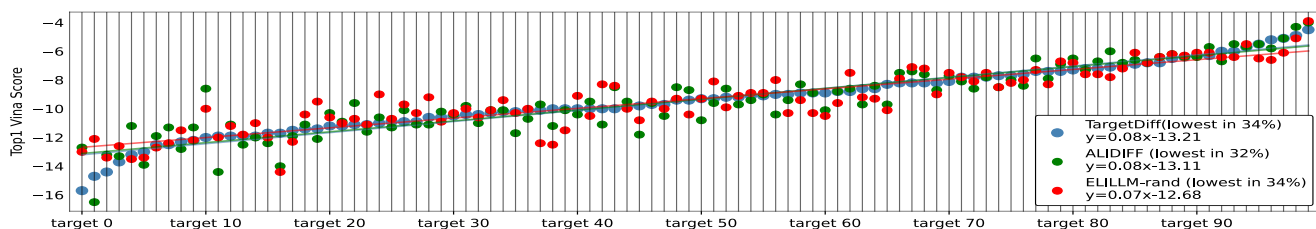


Figure 2: Top1 Vina docking score for different generated molecules (TargetDiff, ALIDIFF, ELILLM-rand) across 100 testing targets, sorted by Vina docking score of TargetDiff result. Trend lines are least-squares linear fits for each method.

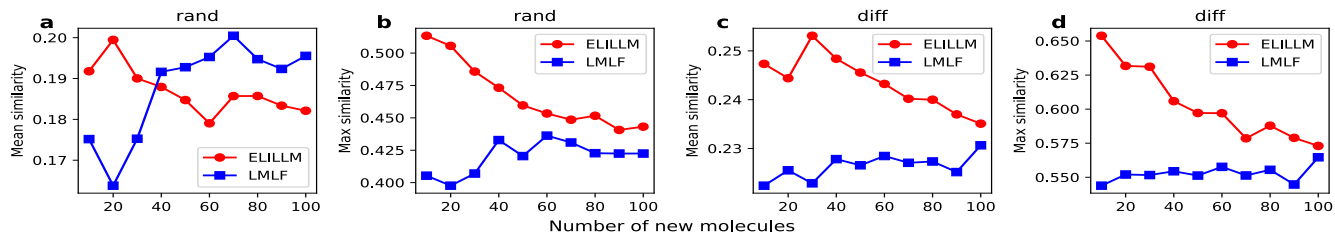


Figure 3: Average Tanimoto similarity between every 10 newly generated molecules and the initial $\mathcal{D}_{\text{obs}}$, including both the average of all pairwise similarities (*mean similarity*) and the average of per-molecule maximum similarity to $\mathcal{D}_{\text{obs}}$ (*max similarity*). Results are averaged across 100 targets to show overall trends under two $\mathcal{D}_{\text{obs}}$ construction strategies.

dataset $\mathcal{D}_{\text{obs}}$: (1) randomly selecting 100 ligands from the training set, docking them against the test targets, and using the resulting ligand–score pairs; and (2) directly adopting the sampled ligands and their docking scores against the 100 test targets as provided by ALIDIFF [9]. The first setting, referred to as **ELILLM-rand**, is designed to evaluate the effectiveness of our method when initialized from a random chemical space. The second setting, termed **ELILLM-diff**, demonstrates that our framework can be effectively integrated with existing deep learning–based generation approaches. Meanwhile, based on the conclusions of Sadeghi et al. [28], we choose to use LLaMAs 3.1 as the basic LLM underlying both our method and related baseline approaches.

4.1.2 Baselines. We compare ELILLM with the following baselines: **Random Init** refers to the randomly constructed $\mathcal{D}_{\text{obs}}$ mentioned earlier, which is used by both ELILLM-rand and LMLF-rand. **liGAN** [26] is a conditional VAE model based on 3D voxelized representations. **AR** [23] and **Pocket2Mol** [25] are 3D autoregressive models at the atom level. **TargetDiff** [10] and **ALIDiff** [9] are diffusion-based approaches designed for target-conditioned molecular generation. Among them, ALIDiff is a recent SOTA method that fine-tunes a pretrained diffusion model through preference optimization. **TamGen** [42] is a GPT-based chemical language model that generates ligand SMILES conditioned on a given target protein. **LMLF** [4] is a method that leverages logical feedback to constrain

a general-purpose LLM for SBDD, producing ligand molecules in the form of SMILES string under the given $\mathcal{D}_{\text{obs}}$. Correspondingly, we also implement a variant of LMLF, referred to as **LMLF-diff**.

4.1.3 Evaluation Metrics. To evaluate the quality of generated molecules in the context of virtual screening, we generate 100 candidate molecules for each of the 100 target proteins in the test set. The generated molecules are evaluated primarily in terms of binding affinity, estimated by the Vina docking score, which directly reflects the LLM’s targeted generation ability. Importantly, in our experiments, only the Vina docking score is used as the optimization guidance, while auxiliary metrics, including drug-likeness (QED) [3], reversed normalized synthetic accessibility (SA) [6], and molecular diversity, are reported for reference to illustrate general molecular properties. Internal diversity, which can be simply measured as the average pairwise Tanimoto distance between generated molecules, may slightly decrease when exploration is concentrated in regions of high docking affinity. In our experiments, the diversity remains above 0.5, which is considered a reasonable range to ensure meaningful coverage of chemical space. QED and SA often exhibit a negative correlation with docking scores and therefore cannot serve as direct indicators of targeted binding affinity.

To ensure a fair comparison, we eliminate the influence of varying 3D coordinates. 3D deep generative models may produce different conformations for the same molecule, potentially leading to inconsistent docking scores. Therefore, we extract the SMILES representation of each generated molecule and reconstruct its 3D conformation using RDKit’s rule-based coordinate generation. The conformations are then optimized via a molecular force field to improve geometric quality. The resulting structures are used for docking with smina to estimate binding affinity scores. In this setup, by controlling the random seed, we ensure that the same SMILES corresponds to the same Vina Docking Score.

Due to space limitations, more detailed experimental settings are provided in Appendix C.

4.2 Performance of SBDD

To answer Q1, we compare ELILLM with baseline methods in a simplified virtual screening scenario, aiming to identify candidate sets of size 1, 5, 10, and 20 with the lowest Vina docking scores for subsequent lead optimization. The results are presented in Table 1. As shown, **ELILLM-rand** outperforms all baseline methods in SBDD, achieving superior performance under the Top 5, 10, and 20 candidate settings despite using only 100 randomly selected $\mathcal{D}_{\text{obs}}$ samples from the full 65K training dataset. While ELILLM-rand demonstrates strong performance, ELILLM-diff further achieves SOTA binding affinity by integrating our framework with a pretrained diffusion model. Specifically, ALIDiff serves as the base generative model. ELILLM-diff outperforms ALIDiff by 4.59%, 5.88%, 6.69%, and 7.77% in the Top 1, 5, 10, and 20 candidate settings, respectively, in terms of binding affinity, showcasing the strong compatibility and enhancement our framework brings to existing deep generative approaches. Moreover, as shown in Appendix Table S1, a Wilcoxon signed-rank test significantly rejects the one-sided hypotheses that ELILLM-diff is weaker than TargetDiff and ALIDIFF ($p < 0.01$), indicating that our results are statistically significant.

Notably, although LMLF achieves lower binding affinity compared to the provided molecules, it shows consistently strong performance in terms of QED, SA, and molecular diversity. This pattern reflects a commonly observed empirical trade-off: QED and SA scores often exhibit a negative correlation with binding affinity. From an encoder–decoder perspective, QED, SA, and diversity fall within the part of the task that LLMs can handle natively via their internal representations and decoding strategies. In contrast, optimizing docking affinity corresponds to the component that lies beyond the LLM’s direct capability and is addressed by our ELILLM framework, which performs controlled exploration in the latent space guided by domain knowledge. The entities generated through this exploration effectively act as intermediate reasoning steps that augment the LLM’s decoding, enabling more targeted generation of high-affinity molecules. These results underscore the importance of incorporating domain knowledge to guide the reasoning and exploration process in the LLM latent space.

As shown in Figure 2, although the mean Top 1 Vina docking score of ELILLM-rand is slightly higher than those of ALIDIFF and TargetDiff, the three methods actually demonstrate comparable performance under this setting. We further observe that the two diffusion-based methods produce nearly identical trend lines, likely due to reliance on the same training distribution. By contrast, our methods, conditioning on a small number of molecules, enable more flexible generation with potential advantages on targets possibly dissimilar to the training set (e.g., Targets 79 and 96).

4.3 Performance of Chemical Space Exploration

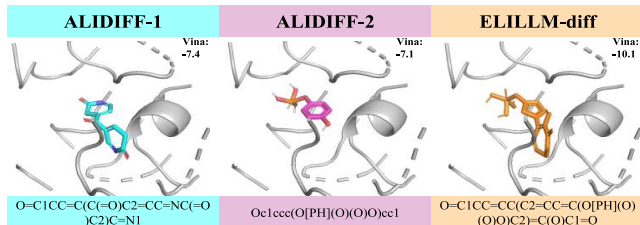
To answer Q2, we aim to evaluate whether our method progressively explores the chemical space or merely remains confined to the region anchored by the initial $\mathcal{D}_{\text{obs}}$. We measure the distance between generated molecules and the initial space using changes in Tanimoto similarity[2] during the iterative generation process. Figure 3 illustrates the chemical space exploration capabilities of our method and LMLF in two settings. It is observed that our method generates molecules progressively farther from the initial space, whereas LMLF is significantly limited to the initial region and even tends to move closer over iterations, despite achieving the highest diversity. Furthermore, by jointly examining subfigures (a) and (b), we observe that in the early stages of generation, our method exhibits an increase in mean similarity while the max similarity decreases, followed by a consistent downward trend in both metrics. These observations indicate that ELILLM behaves like a human expert: initially leveraging existing knowledge to design new molecules by moving away from the most similar instances and instead exploring directions aligned with other diverse examples. It then dynamically incorporates newly acquired knowledge to discover new directions for molecular design.

4.4 Ablation Study

To better understand the contribution of each component in our framework, we conduct an ablation study on the first 10 test targets using ALIDiff-generated $\mathcal{D}_{\text{obs}}$ as initial observations. We evaluate four variants: w/o guide, w/o position, w/o role, and w/o knowledge. The w/o guide variant randomly samples $\mathcal{Z}_{\text{cand}}$ from $\mathcal{Z}_{\text{explore}}$; w/o position uses simple average aggregation with concatenated

Table 2: Ablation study on the first 10 targets from the test set using ALIDIFF-generated $\mathcal{D}_{\text{obs}}$

Methods	Top1 Dock ↓		Top20 Dock ↓		Diversity	
	Avg.	Med.	Avg.	Med.	Avg.	Med.
ELILLM	-9.86	-9.86	-8.79	-8.71	0.69	0.69
w/o guide	-9.08	-9.08	-8.01	-7.90	0.76	0.75
w/o position	-9.80	-9.80	-8.52	-8.38	0.73	0.72
w/o knowledge	-9.40	-9.40	-8.30	-8.04	0.74	0.74
w/o role	-8.20	-8.20	-	-	0.48	0.60
ALIDIFF	-9.10	-9.10	-8.17	-8.04	0.75	0.72

**Figure 4: Visualizations of generated ligands for protein pocket 2jgg generated by ALIDIFF and ELILLM-diff.**

vectors; w/o role has the LLM output the input string directly; and w/o knowledge includes role information only, without chemical knowledge.

As shown in Table 2, w/o guide fails to achieve targeted generation, highlighting the importance of our guidance strategy. w/o position performs slightly worse, confirming that position-aware aggregation captures latent-space patterns more effectively. w/o role exhibits unacceptable performance, generating fewer than 20 distinct molecules across all 10 targets, emphasizing the necessity of role-specific prompting for effective exploration. w/o knowledge shows weaker targeted generation but higher diversity, indicating that prior knowledge helps constrain LLM decoding towards high-affinity regions.

4.5 Visualization

We visualize the generated ligand for the protein pocket 2jgg to further demonstrate our expert-like exploratory generation process. As shown in Figure 4, without any human guidance (such as identifying key functional groups or scaffolds), our method explores the latent space and designs a new molecule that combines the “O=C1CC=C” fragment from ligand 1 and the “O[PH](O)(O)O” group from ligand 2, and completes the rest using other learned structural knowledge.

5 Related Work

LLM generation as encoder-decoder pipelines. LLM-based generation can be viewed as an encoder-decoder process, where the encoder maps inputs into latent representations and the decoder autoregressively generates outputs. Prior work mainly enhances either the encoder, e.g., through knowledge-enhanced pretraining or soft prompts [16, 33, 36, 46], or the decoder via prompt engineering and feedback-based refinement [4, 14, 27, 38]. In contrast, our method enables explicit exploration and control within the latent

space constructed by the encoder, supporting more structured and efficient scientific generation.

SBDD and LLM. With the increasing availability of protein-ligand structural data, structure-based drug design (SBDD) has become a central application scenario for generative molecular modeling, with a wide range of 3D-aware generative approaches proposed in prior work. LLMs have shown promising capabilities across a variety of tasks beyond molecular generation, including reasoning, graph-based learning, and iterative feedback-guided optimization [12, 27, 43]. These successes highlight the versatility of pretrained LLMs as knowledge sources that can be leveraged without task-specific finetuning. Building on this idea, models such as MolGPT [1], LMLF [4], Token-Mol [35], TamGen [42] and DrugLLM [22] leverage textual and structural representations learned during pretraining to generate molecules. However, these methods often require task-specific finetuning or large domain datasets to capture complex protein-ligand interactions effectively. In contrast, our framework addresses this challenge in a *plug-in* fashion, without retraining the LLM. ELILLM explicitly explores the latent space of pretrained LLMs using domain knowledge and a position-aware surrogate model to handle aspects of SBDD that the model alone cannot fully capture. By performing controlled latent space exploration, ELILLM generates candidate embeddings that augment the LLM’s decoding process, enabling targeted molecular generation while preserving the pretrained knowledge.

6 Conclusion

In this work, we present ELILLM, a novel exploration-augmented latent inference framework tailored for SBDD. ELILLM reformulates the LLM generation process into a three-stage pipeline of encoding, exploration, and decoding, explicitly leveraging the expressive power of latent spaces to address the challenge of systematic latent space navigation. We instantiate this framework for SBDD with three key components: Molecular-Guided LLM Encoding, BO-based Exploration, and Knowledge-Guided LLM Decoding.

Our evaluation on the CrossDocked2020 benchmark demonstrates that ELILLM achieves superior performance in binding affinity scores, surpassing seven baseline methods. Importantly, ELILLM shows strong compatibility with pretrained models, enabling it to be seamlessly integrated with existing LLM-based molecular generation approaches. By simulating human-like research workflows with interpretability and efficiency, ELILLM offers a principled and effective approach for optimizing docking-based molecular design.

Future work will focus on generalizing the proposed techniques to broader scenarios beyond SBDD. In particular, we aim to explore applications in graph representation learning [44, 45] and recommender systems [19, 20], where latent space exploration and uncertainty-aware modeling could provide similar benefits.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant Nos. U22A2098, 62172185, 62206105, 62202200, and 62406127; the KeyScience and Technology Development Plan of Jilin Province under Grant No.20240302078GX.

References

- [1] Viraj Bagal, Rishal Aggarwal, PK Vinod, and U Deva Priyakumar. 2021. MolGPT: molecular generation using a transformer-decoder model. *Journal of chemical information and modeling* 62, 9 (2021), 2064–2076.
- [2] Dávid Bajusz, Anita Rácz, and Károly Héberger. 2015. Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of cheminformatics* 7 (2015), 1–13.
- [3] G Richard Bickerton, Gaia V Paolini, Jérémy Besnard, Sorel Muresan, and Andrew L Hopkins. 2012. Quantifying the chemical beauty of drugs. *Nature chemistry* 4, 2 (2012), 90–98.
- [4] Shreyas Bhat Brahmavar, Ashwin Srinivasan, Tirtharaj Dash, Sowmya Ramaswamy Krishnan, Lovekesh Vig, Arijit Roy, and Raviprasad Aduri. 2024. Generating novel leads for drug discovery using LLMs with logical feedback. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21–29.
- [5] Vijil Chenthamarakshan, Payel Das, Samuel Hoffman, Hendrik Strobelt, Inkit Padhi, Kar Wai Lim, Benjamin Hoover, Matteo Manica, Jannis Born, Teodoro Laino, et al. 2020. CogMol: Target-specific and selective drug design for COVID-19 using deep generative models. *Advances in Neural Information Processing Systems* 33 (2020), 4320–4332.
- [6] Peter Ertl and Ansgar Schuffenhauer. 2009. Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *Journal of cheminformatics* 1 (2009), 1–11.
- [7] Paul G Francoeur, Tomohide Masuda, Jocelyn Sunseri, Andrew Jia, Richard B Iovanisci, Ian Snyder, and David R Koes. 2020. Three-dimensional convolutional neural networks and a cross-docked data set for structure-based drug design. *Journal of chemical information and modeling* 60, 9 (2020), 4200–4215.
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [9] Siyi Gu, Minkai Xu, Alexander Powers, Wei Nie, Tomas Geffner, Karsten Kreis, Jure Leskovec, Arash Vahdat, and Stefano Ermon. 2024. Aligning target-aware molecule diffusion models with exact energy optimization. *Advances in Neural Information Processing Systems* 37 (2024), 44040–44063.
- [10] Jiaqi Guan, Wesley Wei Qian, Xingang Peng, Yufeng Su, Jian Peng, and Jianzhu Ma. 2023. 3D Equivariant Diffusion for Target-Aware Molecule Generation and Affinity Prediction. In *International Conference on Learning Representations*.
- [11] Jiaqi Guan, Xiangxin Zhou, Yuwei Yang, Yu Bao, Jian Peng, Jianzhu Ma, Qiang Liu, Liang Wang, and Quanquan Gu. 2023. DecompDiff: Diffusion Models with Decomposed Priors for Structure-Based Drug Design. In *International Conference on Machine Learning*. PMLR, 11827–11846.
- [12] Jiao Huang, Qianli Xing, Jinglong Ji, and Bo Yang. 2025. Code-Generated Graph Representations Using Multiple LLM Agents for Material Properties Prediction. In *Forty-second International Conference on Machine Learning*.
- [13] Zhilin Huang, Ling Yang, Xiangxin Zhou, Zhilong Zhang, Wentao Zhang, Xiaowu Zheng, Jie Chen, Yu Wang, Bin Cui, and Wenming Yang. 2024. Protein-ligand interaction prior for binding-aware 3d molecule diffusion models. In *The Twelfth International Conference on Learning Representations*.
- [14] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [15] Agustinus Kristiadi, Felix Strieth-Kalthoff, Marta Skreta, Pascal Poupart, Alan Aspuru-Guzik, and Geoff Pleiss. 2024. A Sober Look at LLMs for Material Discovery: Are They Actually Good for Bayesian Optimization Over Molecules?. In *International Conference on Machine Learning*. PMLR, 25603–25622.
- [16] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
- [17] Anchen Li, Elena Casiraghi, and Juho Rousu. 2024. Chemical reaction enhanced graph learning for molecule representation. *Bioinformatics* 40, 10 (2024), btad558.
- [18] Anchen Li, Elena Casiraghi, and Juho Rousu. 2025. CSGL: chemical synthesis graph learning for molecule representation. *Bioinformatics* 41, 7 (2025), btad555.
- [19] Anchen Li, Jinglong Ji, Riting Xia, and Bo Yang. 2025. Diffusion Multi-behavior Recommender Model. In *International Conference on Database Systems for Advanced Applications*. Springer, 202–217.
- [20] Anchen Li, Bo Yang, Huan Huo, Farookh Hussain, and Guandong Xu. 2025. Hypercomplex knowledge graph-aware recommendation. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval*. 2017–2026.
- [21] Meng Liu, Youzhi Luo, Kanji Uchino, Koji Maruhashi, and Shuiwang Ji. 2022. Generating 3D Molecules for Target Protein Binding. In *International Conference on Machine Learning*. PMLR, 13912–13924.
- [22] Xianggen Liu, Yan Guo, Haoran Li, Jin Liu, Shudong Huang, Bowen Ke, and Jiancheng Lv. 2024. Drugllm: Open large language model for few-shot molecule generation. *arXiv preprint arXiv:2405.06690* (2024).
- [23] Shitong Luo, Jiaqi Guan, Jianzhu Ma, and Jian Peng. 2021. A 3D generative model for structure-based drug design. *Advances in Neural Information Processing Systems* 34 (2021), 6229–6239.
- [24] Sergei Manzhos and Manabu Ihara. 2023. The loss of the property of locality of the kernel in high-dimensional Gaussian process regression on the example of the fitting of molecular potential energy surfaces. *The Journal of Chemical Physics* 158, 4 (2023).
- [25] Xingang Peng, Shitong Luo, Jiaqi Guan, Qi Xie, Jian Peng, and Jianzhu Ma. 2022. Pocket2mol: Efficient molecular sampling based on 3d protein pockets. In *International Conference on Machine Learning*. PMLR, 17644–17655.
- [26] Matthew Ragoza, Tomohide Masuda, and David Ryan Koes. 2022. Generating 3D molecules conditional on receptor binding sites with deep generative models. *Chemical science* 13, 9 (2022), 2701–2713.
- [27] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilian Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. 2024. Mathematical discoveries from program search with large language models. *Nature* 625, 7995 (2024), 468–475.
- [28] Shaghayegh Sadeghi, Alan Bui, Ali Forooghi, Jianguo Lu, and Alioune Ngom. 2024. Can large language models understand molecules? *BMC bioinformatics* 25, 1 (2024), 225.
- [29] Yiqing Shen, Zan Chen, Michail Mamalakis, Luhan He, Haiyang Xia, Tianbin Li, Yanzhou Su, Junjun He, and Yu Guang Wang. 2024. A fine-tuning dataset and benchmark for large language models for protein understanding. In *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2390–2395.
- [30] Niranjan Srinivas, Andreas Krause, Sham Kakade, and Matthias Seeger. 2010. Gaussian Process Optimization in the Bandit Setting: No Regret and Experimental Design. In *Proceedings of the 27th International Conference on Machine Learning*. Omnipress, 1015–1022.
- [31] Michael L Stein. 1999. *Interpolation of spatial data: some theory for kriging*. Springer Science & Business Media.
- [32] Haotian Sun, Yuchen Zhuang, Linghai Kong, Bo Dai, and Chao Zhang. 2023. Adaplaner: Adaptive planning from feedback with language models. *Advances in neural information processing systems* 36 (2023), 58202–58245.
- [33] Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou, and Daniel Cer. 2022. SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 5039–5059. doi:10.18653/v1/2022.acl-long.346
- [34] Haiming Wang, Huajian Xin, Chuanyang Zheng, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, et al. 2024. LEGO-Prover: Neural Theorem Proving with Growing Libraries. In *12th International Conference on Learning Representations (ICLR 2024)*. International Conference on Learning Representations, ICLR.
- [35] Jike Wang, Rui Qin, Mingyang Wang, Meijing Fang, Yangyang Zhang, Yuchen Zhu, Qun Su, Qiaolin Gou, Chao Shen, Odin Zhang, et al. 2025. Token-Mol 1.0: tokenized drug design with large language models. *Nature Communications* 16, 1 (2025), 1–19.
- [36] Xiaozhi Wang, Tianyu Gao, Zhaocheng Zhu, Zhengyan Zhang, Zhiyuan Liu, Juanzi Li, and Jian Tang. 2021. KEPLER: A unified model for knowledge embedding and pre-trained language representation. *Transactions of the Association for Computational Linguistics* 9 (2021), 176–194.
- [37] Ye Wang, Honggang Zhao, Simone Scibola, and Wenlu Wang. 2023. cMolGPT: a conditional generative pre-trained transformer for target-specific de novo molecular generation. *Molecules* 28, 11 (2023), 4430.
- [38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [39] Frank Wilcoxon. 1992. Individual comparisons by ranking methods. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 196–202.
- [40] Christopher KI Williams and Carl Edward Rasmussen. 2006. *Gaussian processes for machine learning*. Vol. 2. MIT press Cambridge, MA.
- [41] Andrew Gordon Wilson, Zhiting Hu, Ruslan Salakhutdinov, and Eric P Xing. 2016. Deep kernel learning. In *Artificial intelligence and statistics*. PMLR, 370–378.
- [42] Kehan Wu, Yingce Xia, Pan Deng, Renhe Liu, Yuan Zhang, Han Guo, Yumeng Cui, Qizhi Pei, Lijun Wu, Shufang Xie, et al. 2024. TamGen: drug design with target-aware molecule generation through a chemical language model. *Nature Communications* 15, 1 (2024), 9360.
- [43] Riting Xia, Rucong Wang, Yulin Liu, Anchen Li, Xueyan Liu, and Yan Zhang. 2025. When Noisy Labels Meet Class Imbalance on Graphs: A Graph Augmentation Method with LLM and Pseudo Label. *arXiv preprint arXiv:2507.18153* (2025).
- [44] Riting Xia, Chunxu Zhang, Xueyan Liu, Anchen Li, and Yan Zhang. 2025. GraphLAM: Two-Stage Algorithm for Improving Class-Imbalanced Node Classification on Attribute-Missing Graphs. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 3478–3487.
- [45] Xuan Zang, Bo Yang, Xueyan Liu, and Anchen Li. 2021. Dnea: Dynamic network embedding method for anomaly detection. In *International conference on*

knowledge science, engineering and management. Springer, 236–248.

- [46] Taolin Zhang, Chengyu Wang, Nan Hu, Minghui Qiu, Chengguang Tang, Xiaofeng He, and Jun Huang. 2022. DKPLM: decomposable knowledge-enhanced pre-trained language model for natural language understanding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 11703–11711.

A Proof

A.1 Proof of Theorem 3.1

PROOF. By definition, the embedding of token t is given by

$$\tilde{z}_t = E[t],$$

which depends solely on the token identity t and does not involve the positional index p_t .

Formally, for any measurable function f of the embedding,

$$\mathbb{P}(f(\tilde{z}_t) \leq x | p_t) = \mathbb{P}(f(\tilde{z}_t) \leq x),$$

because the distribution of $\tilde{z}_t$ is completely determined by t and does not involve p_t .

Hence, the embedding $\tilde{z}_t$ is independent of its positional index p_t . $\square$

A.2 Proof of Theorem 3.2

PROOF. Let $\mu = \mathbb{E}[\tilde{z}_{i,t}^{(j)}]$ denote the mean embedding vector. The aggregation is defined as

$$\tilde{z}_i^{(j)} = \frac{1}{n_i} \sum_{t=1}^{n_i} \frac{\text{Concat}(p_t \tilde{z}_{i,t}^{(j)}, (l_{\max} - p_t) \tilde{z}_{i,t}^{(j)})}{l_{\max}}.$$

Since the aggregation is linear in $\tilde{z}_{i,t}^{(j)}$ and by Theorem 3.1 the embeddings are independent of p_t , taking expectation gives

$$\mathbb{E}[\tilde{z}_i^{(j)}] = \frac{1}{n_i l_{\max}} \sum_{t=1}^{n_i} \text{Concat}(p_t \mu, (l_{\max} - p_t) \mu).$$

Here, the expected aggregated vector depends only on the positional indices p_t through the linear coefficients, and not on the actual embedding values. Therefore, each position contributes proportionally in expectation, and no single position p_t receives undue influence. This establishes that the aggregation is statistically unbiased with respect to position. $\square$

B Appendix For Method Details

B.1 Algorithm Pseudocode

We present the pseudocode of ELILLM in Algorithm 1.

B.2 GP Kernel Details

In our surrogate model $\hat{f}_\theta$, we adopt a composite kernel that combines the Matérn-1.5 and Matérn-2.5 kernels [31] to better capture both local variations and global smoothness in the latent space. The resulting kernel is defined as:

$$k(\mathbf{z}, \mathbf{z}') = \lambda_1 \cdot k_{\text{Matern-1.5}}(\mathbf{z}, \mathbf{z}') + \lambda_2 \cdot k_{\text{Matern-2.5}}(\mathbf{z}, \mathbf{z}'),$$

where λ_1 and λ_2 are learnable weights.

The Matérn-1.5 kernel is given by

$$k_{\text{Matern-1.5}}(\mathbf{z}, \mathbf{z}') = \sigma^2 \left(1 + \frac{\sqrt{3} \|\mathbf{z} - \mathbf{z}'\|}{\ell} \right) \exp \left(-\frac{\sqrt{3} \|\mathbf{z} - \mathbf{z}'\|}{\ell} \right),$$

and the Matérn-2.5 kernel is

$$k_{\text{Matern-2.5}}(\mathbf{z}, \mathbf{z}') = \sigma^2 \left(1 + \frac{\sqrt{5} \|\mathbf{z} - \mathbf{z}'\|}{\ell} + \frac{5 \|\mathbf{z} - \mathbf{z}'\|^2}{3\ell^2} \right) \exp \left(-\frac{\sqrt{5} \|\mathbf{z} - \mathbf{z}'\|}{\ell} \right)$$

where ℓ is the length-scale and σ^2 is the output variance, both of which are learnable hyperparameters. The combination of Matérn-1.5 and Matérn-2.5 allows the model to benefit from the flexibility of the former in capturing sharp local transitions and the smoothness of the latter in modeling broader trends.

B.3 Prompt

We present the prompts used in our study in Figures S1–S3: Figure S1 shows the prompt designed for Knowledge-Guided LLM Decoding, Figure S2 shows the prompt used in the w/o knowledge ablation study, and Figure S3 shows the prompt used in the w/o role ablation setting.

C Detailed Experimental Settings

C.1 Hyperparameter Settings

For our method, we explored a series of hyperparameter ranges across architecture design, surrogate model training, and generation-time settings.

In terms of **architecture**, we adopted an MLP structure of 8192-256-256-256-20, corresponding to three hidden layers. The hidden layer size was searched over $\{128, 256, 512\}$, and 256 was selected as the final choice. The latent dimension was chosen from $\{10, 15, 20, 50\}$, with 20 providing the best performance. For the Gaussian Process (GP) component, we tested several kernel types including RBF, linear, Matérn, and their combinations, and finally selected a hybrid of Matérn-1.5 and Matérn-2.5 kernels. For **surrogate model training**, the MLP was trained for 100 epochs in ELILLM-rand and 200 epochs in ELILLM-diff, while the GP model was trained for 100 epochs. The learning rate for the MLP was searched over $\{0.001, 0.005, 0.01\}$, and 0.001 was chosen. For the GP, the learning rate was tuned within $\{0.05, 0.1\}$, and 0.1 was selected. The Adam optimizer was used throughout training. Regarding **generation-time parameters**, the perturbation scale λ_{perturb} was tuned over $\{0.3, 0.4, 0.5\}$, with 0.4 adopted as the final value. The maximum token length $l_{\max}$ was selected from $\{60, 80, 100\}$, and set to 80. The number of candidates $n_{\text{candidate}}$ was fixed at 5. The sampling temperature of the LLM was tuned over $\{0.4, 0.5\}$, with 0.4 selected, and the acquisition function threshold δ was set to 0.1. All LLM-based generation was performed using a locally deployed LLaMA 3.1 8B model.

For the baselines, we followed the default hyperparameters provided in their original papers or official implementations. To ensure a fair comparison, we re-implemented LMLF based on LLaMA 3.1 8B. To guarantee that LMLF can run successfully on the task of generating 100 ligands, we set the *docking threshold* to -6.5 , and the *threshold increment frequency* to 50, meaning the threshold decreases by 1 every 50 iterations. In line with our setup, the number of ligands generated per round was capped at 5. All generated molecules, including those from baselines, were evaluated using our docking pipeline to ensure fair comparison. Specifically, we used RDKit to parse the generated SMILES strings, construct initial 3D

Algorithm 1 Procedure of ELILLM

Input: Observed dataset $\mathcal{D}_{\text{obs}} = \{(s_i, y_i)\}_{i=1}^n$, prompt p , protein pocket t , generation count k
Output: Generated molecule set $\mathcal{S}_{\text{out}}$

```

1: Initialize  $\mathcal{S}_{\text{out}} \leftarrow \emptyset$ 
2: while  $|\mathcal{S}_{\text{out}}| < k$  do
3:   Update latent observations  $\mathcal{Z}_{\text{obs}}$  using Equation 7                                /** Project  $\mathcal{D}_{\text{obs}}$  into latent embedding space **/
4:   Train surrogate model  $\hat{f}_{\theta}$  on  $\{(z_i, y_i)\}$  using Equations 14 and 15          /** Fit surrogate model based on latent embeddings **/
5:   Sample exploration embedding set  $\mathcal{Z}_{\text{explore}}$  using Equations 8 and 9              /** Generate candidate embeddings in latent space **/
6:   Simulate evaluation of  $\mathcal{Z}_{\text{explore}}$  using surrogate model with Equation 12         /** Simulated assessment via surrogate model **/
7:   Select candidate embedding set  $\mathcal{Z}_{\text{cand}}$  based on Equation 18                     /** Choose promising embeddings for generation **/
8:   Decode candidates to molecules using Equation 19                               /** Generate candidate molecules from embeddings **/
9:   Check validity and remove duplicates in  $\mathcal{S}_{\text{cand}}$                                 /** Filter invalid or repeated molecules **/
10:  Evaluate filtered  $\mathcal{S}_{\text{cand}}$  with docking score  $y = \text{Dock}(s, t)$                  /** Compute true docking scores using molecule  $s$  and protein pocket  $t$  **/
11:  Update  $\mathcal{S}_{\text{out}} \leftarrow \mathcal{S}_{\text{out}} \cup \mathcal{S}_{\text{cand}}$                                 /** Add new valid molecules to output set **/
12:  Update  $\mathcal{D}_{\text{obs}} \leftarrow \mathcal{D}_{\text{obs}} \cup \{(s, y) \mid s \in \mathcal{S}_{\text{cand}}, y = \text{Dock}(s, t)\}$  /** Augment observed data with new molecules and docking scores **/
13: end while
14: return  $\mathcal{S}_{\text{out}}$ 

```

ELILLM Prompt

Instructions: You are a SMILES repair engine that must always generate the closest valid SMILES approximation for any input, never returning an empty string. You may use the following rules as appropriate, including but not limited to:

Basic Repairs:

Complete missing ring numbers: C1CC $\rightarrow$ C1CC1

Balance parentheses: C(=O)(O $\rightarrow$ C(=O)O

Fix atom formatting: na $\rightarrow$ [Na]

Error Tolerance:

Illegal characters $\rightarrow$ replace with carbon or other atoms that you think are most likely: [X]C $\rightarrow$ CC

Uncloseable rings $\rightarrow$ convert to chains: C1CC2 $\rightarrow$ CCC

Hypervalent atoms $\rightarrow$ remove excess bonds: C(C)(C)(C)(C) $\rightarrow$ C(C)(C)C

Final Safeguard:

When completely unrecognizable: Keep all valid atoms and single bonds: A*B $\rightarrow$ CB

Minimal output: ./. $\rightarrow$ C

Examples:

(ring closure)Input: C1CC $\rightarrow$ Output: C1CC1

(invalid atom $\rightarrow$ C)Input: [X][Na+] $\rightarrow$ Output: C[Na+]

(remove unparseable stereochemistry)Input: C(/N)=C/F $\rightarrow$ Output: C(N)=CF

(wildcard $\rightarrow$ C)Input: *C(=O)O $\rightarrow$ Output: CC(=O)O (keep only valid atoms)Input: A1B2 $\rightarrow$ Output: CC

Output Specification:

Only return the repaired SMILES string, nothing else. No explanations, no confirmations, just the valid SMILES. If input is already valid, return it unchanged.

Figure S1: ELILLM prompt

Table S1: One-sided Wilcoxon signed-rank test p -values assessing the statistical significance that ELILLM-diff achieves significantly lower docking scores compared to some baseline methods across Top-K results

Methods	Top1 Dock p	Top5 Dock p	Top10 Dock p	Top20 Dock p
LiGAN	1.33e-14	5.70e-15	5.40e-15	1.42e-14
Pocket2Mol	2.71e-5	1.70e-4	4.37e-4	1.45e-3
TargetDiff	7.72e-4	1.21e-6	3.85e-8	1.01e-9
ALIDIFF	1.13e-5	1.19e-9	3.41e-12	6.50e-14

w/o knowledge Prompt

Output a SMILES string only!
 You are a SMILES repair engine that must always generate the closest valid SMILES approximation for any input, never returning an empty string.
 If the input string is already a valid SMILES, output this SMILES only!!

Figure S2: w/o knowledge prompt**w/o role Prompt**

Output a SMILES string only!
 Input a SMILES string to you, and your task is to output this SMILES string.

Figure S3: w/o role prompt

coordinates via molecular embedding, and optimize them using the MMFF94 force field. Vina docking scores were then computed using Smina with the default scoring function (vina). The exhaustiveness was set to 32, the number of poses was kept at the default value of 9, and the binding pocket was determined based on the known co-crystallized ligand of the target protein. The autobox-add parameter was set to 1 Å to define the docking box around the pocket. Among all generated poses, we selected the one with the lowest docking score as the final Vina score.

C.2 System and Software Configuration

The experiments under the **ELILLM-rand** and **ELILLM-diff** settings were conducted on two separate machines, both equipped with **NVIDIA RTX 3090 GPUs**.

For the **ELILLM-rand** setting, the system was configured with an Intel(R) Xeon(R) CPU E5-2673 v4 @ 2.30GHz and ran on Ubuntu 20.04.6 LTS. For the **ELILLM-diff** setting, the hardware setup included an Intel(R) Xeon(R) Silver 4310 CPU @ 2.10GHz, also running Ubuntu 20.04.6 LTS.

Both experiments shared the same major software environment, including Python 3.10, PyTorch 2.3.1, HuggingFace Transformers 4.51.3, RDKit 2024.9.6, Smina 2020.12.10, and CUDA 12.1.

C.3 Code and Reproducibility

To ensure reproducibility, we include all code and scripts necessary for training, evaluation, and visualization in an anonymous online repository at <https://github.com/hxn timer/ELILLM>. All experiments are conducted under controlled settings with fixed random seeds (e.g., seed = 1) to minimize randomness and ensure consistency.

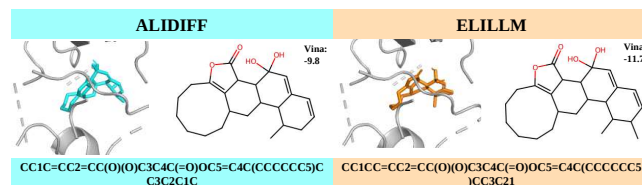
D Supplementary Experiment**D.1 One-sided Wilcoxon Signed-Rank Test**

To assess the statistical significance of our method's performance improvement, we conduct one-sided Wilcoxon signed-rank [39] tests on the average docking scores (lower is better) across 100

target proteins under Top-1, Top-5, Top-10, and Top-20 candidate settings. Specifically, we compare our method against several strong baseline methods with the best reported performance.

The null hypothesis for each test is that our method does not achieve significantly lower docking scores than the compared baseline (i.e., the paired differences are symmetrically distributed around zero or skewed in the opposite direction). The alternative hypothesis is that our method achieves significantly lower scores, indicating superior binding affinity.

The test is non-parametric and appropriate for our setting where paired scores are compared across the same set of protein targets, but normality of differences cannot be assumed. Reported *p*-values reflect the confidence with which we can reject the null hypothesis and support the superiority of our method. As shown in Table S1, the *p*-values in all settings are below the 0.05 significance threshold, indicating that our method is significantly better than the selected baselines across all Top-*K* docking score evaluations.

D.2 Additional Visualization**Figure S4: Visualizations of generated ligands for protein pockets 3nfb generated by ALIDIFF and ELILLM-diff.**

While the visualizations in the main paper highlight the exploration capability of ELILLM, its ability to exploit existing molecular structures is equally important. As shown in Figure S4, ELILLM achieves substantial improvements in binding affinity by making slight modifications to the original structure while preserving its overall scaffold.